

BMB 306

YAZILIM MÜHENDİSLİĞİ

Yazılım Kalite Unsurları

☞ Yazılım Kalite Unsurları

- o Kalitenin birçok tanımı yapılmıştır. Bunlardan birkaçını sıralarsak:
 - Kalite amaca uygunluktur.
 - Kalite insanlar için değer taşımaktır.
- o Yazılım kalitesini belirleyen birçok özellik vardır.
- o Yazılım hemen hemen tamamen hatalarından arındırılmış olsa bile kaliteli olmayabilir.
- o Yani hatasızlık bir kalite ölçütüdür ancak kaliteyi belirleyen yegane özellik değildir.
- o Üstelik bu özelliklerden bazıları bir ürün için daha önemli iken bazıları diğer bir ürün için daha önemli olabilir.

Yazılım Kalite Unsurları

Yazılım Kalite Unsurları

- o Yazılım kalitesini belirleyen özellikler büyük ölçüde McCall (1977) ve Boehm'in (1978) çalışmaları ile belirlendi.
- o Bu bilgilerden de faydalanarak yazılım kalitesini belirleyen unsurları şöyle sıralayabiliriz:
 - **Anlaşılabilirlik (Understandability):** Gerek sistem yazılımı, gerekse dokümanlar anlaşılır ve açık mı?
 - **Tamlık (Completeness):** Tüm gerekli bileşenler gerçekleşmiş mi? Sistem kaynakları yeterli mi?
 - **Doğruluk (Correctness):** Sistem kendisinden beklenen işlevleri doğru olarak yerine getiriyor mu?
 - **Taşınabilirlik (Portability):** Yazılım başka bir platforma taşınabiliyor mu? Taşınamıyorsa bu belirtilmiş mi?
 - **Sürdürülebilirlik-Bakım yapılabilirlik (Maintainability):** Yazılımın ileride çıkabilecek sorunları kolayca çözülebiliyor mu?
 - **Test edilebilirlik (Testability):** Yazılım kolayca ve doğru olarak test edilebiliyor mu?
 - **Kullanıma uygunluk (Usability):** Yazılım kullanıcı dostu mu?
 - **Güvenilirlik (Reliability) ve Sağlamlık (Robustness):** Yazılım kendisinden beklenenleri gerek süre gerekse koşullara bağlı olmaksızın doğru olarak yerine getirebiliyor mu?
 - **Etkinlik (Efficiency):** Yazılım (bellek, mikroişlemci kullanımı, haberleşme ağ kullanımı gibi konularda) hem etkin biçimde yazılmış hem de etkin bir biçimde çalışabiliyor mu?

Yazılım Kalite Unsurları

∞ Yazılım Kalite Unsurları (devam)

- **Güvenlik (Security):** Yazılıma sadece yetkisi olan kullanıcılar erişebiliyor mu? Yazılımın kendisi ve verileri yetkisiz erişime karşı korunuyor mu?
- **Esneklik (Flexibility):** Yazılıma gerektiği zaman yeni modüller veya işlevler kolayca eklenebiliyor mu?
- **Tekrar kullanılabilirlik (Reusability):** Yazılımın modülleri veya tamamını başka projelerde kullanmak mümkün mü?
- **Tutarlılık (Consistency):** Yazılımda kullanılan değişken adları, matematiksel veya çeşitli semboller ve ekrandaki elemanlar tutarlı ve birbiri ile uyumlu mu?
- **Farkındalık (Conciseness):** İlgili ürün yazılım kuralları, standartları gözetilerek, yeniden düzenleme ve optimizasyon kurallarına uyularak ve farkında olunarak yazılmış mı?
- **Erişilebilirlik-Edinilebilirlik (Availability):** Yazılıma piyasada veya herhangi bir kanalla erişmek kolay mı?
- **Kurulabilirlik (Installability):** Yazılımı kurmak kolay mı?
- **Uyumluluk (Compatibility):** Yazılım diğer sistemlerle uyumlu olarak çalışabiliyor mu?

"Bir programı güvenli, güvenilir ve hızlı yapmanın tek yöntemi onu küçük yapmaktır."

Andrew S. Tanenbaum

Kaliteli Kodlama

Kaliteli Kodlamada Dikkat Edilecek Noktalar ve İpuçları

Bu kısımda yazılımcılara daha kaliteli kod yazabilmeleri için her seviyeden öneriler bulunmaktadır.

Kaliteli Kodlama

- ✎ **Yazdığınız Her Modül İçin Ayrı Bir Test Programı Yazmalısınız**
 - o Burada modülden kastımız herhangi bir projede kolaylıkla kullanılabilecek bir kütüphane kod birimidir. Örneğin bir dll (Dynamically Linked Library) bir modül olarak kabul edilebilir.
 - o O halde tekrar ana konuya gelecek olursak kodladığınız her modül (örneğin .dll) için bir test programı yazmalı ve modülü bu test programı içinden koşturmalısınız.
 - o Bu test programını yazmak için harcayacağınız süre inanılmaz yazmazsanız daha sonra çıkacak problemleri düzeltmek için harcayacağınız sürenin yanında çok küçük kalır.

Kaliteli Kodlama

🌀 Nasıl Yapılacağını Değil Ne Yapılacağını Kodlayın

- o Kodunuzu gruplarken ne yapılacağına göre kodları gruplayın.
- o Bir işin nasıl yapıldığını (yani detayları) kod içerisine yaymayın. İyi bir kod alışveriş listesi gibi olmalıdır; “Bunu yap”, “Şunu yap”, “Onu yap” şeklinde kodlanmalıdır.
- o Örnek: Aşağıdaki kod birden ona kadar olan sayıların nasıl toplandığının detaylarını gösteriyor, sanırım kodunuzu okuyanlar bu detayları çok da merak etmiyorlardır, ilgilenenler de zaten detaylara bakacaktır.

// Birden herhangi bir sayıya kadar toplamı hesapla

```
int toplam = 0;
for (int i = 1; i <= 10; i++)
{
    toplam = toplam + i;
}
```

Kaliteli Kodlama

☞ Nasıl Yapılacağını Değil Ne Yapılacağını Kodlayın (devam)

- Örnek: Oysa aşağıdaki kodda main işlevine bakarsak her satır sadece ne yapıldığını belirtmekte. İşin nasıl yapıldığının detayları ise bir işlev içine gizlenmiş olarak derli toplu durmaktadır.

```
int BirdenSayiyaKadarToplamDondur(int sayi)
{
    int toplam = 0;
    for (int i= 1; i <= sayi; i++)
    {
        toplam = toplam + i;
    }
    return toplam;
}

int main(void)
{
    int sayilarToplami;
    sayilarToplami = BirdenSayiyaKadarToplamDondur(10);
    EkranaGoster(sayilarToplami);
    YazicidaBastir(sayilarToplami);
}
```

- ☞ “Nesne Tabanlı Programlamada “Encapsulation” kavramı da yukarıda anlattıklarımızı destekler. Detayların gizlenmesi ve işin adının, amacının ön plana çıkarılması günümüz programlamasında esas bir amaçtır.

Kaliteli Kodlama

∞ Yorumların Kaynak Koda Oranı Ne Kadar Olmalı

- o Yorum yazmamanın veya çok az yorum yazmanın kötü bir alışkanlık olduğu zaten bildiğimiz bir konu. Kodda mutlaka ve mutlaka yorum satırları bulunmalıdır.
- o Bunun yanı sıra çok fazla yorum yazmak da çok iyi bir alışkanlık değildir. Çok fazla yorum yazmak zorunda kalıyorsanız bu sizin kodu sağlam, kendi kendini anlatan biçimde yazamadığınız ve bu yüzden sürekli açıklamak zorunda kaldığınız anlamına gelir.
- o Eğer bir kodu sürekli açıklamak zorunda kalıyorsanız o kodda bir sorun var demektir.
- o Üstelik yorumların kod ile bütünlüğünü sağlamak da zaman alıcı ek bir iştir.
- o Kodun bakımının yapıldığı gibi yorumların da bakımı yapılmalıdır.
- o Peki ne kadar yorum satırı olmalı. Tecrübelerle göre kod satırlarının **en az beşte biri kadar** yorum satırı olmalıdır.
- o En fazla da kod satırları sayısı kadar yorum satırı olmalı, yani bir kod satırına karşılık bir yorum satırı olmalı. Bunu açıklayacak olursak önemli gördüğünüz her noktada, işlev başlarında, dosya başlarında mutlaka o kodu yazmaktaki niyeti açıklayan yorum satırları olmalıdır.

Kaliteli Kodlama

☞ Yorumda Ne Yazılır

- o Yorumları programcı olmayan birinin de anlayabileceği şekilde yazın. Neyin neden ve nasıl yapıldığını açıklayan yorumlar yazın.
- o Kodun aynısını kendi dilinizde veya yabancı bir dilde aynen tekrar etmeyin.
- o Yorumlarda o kod bölgesini yazmaktaki niyetinizi, ana amacınızı belirtin.
- o **Tecrübesiz bir programcı:** Bir işlevi veya iş satırlarını göz önüne aldığımızda tecrübesiz bir programcı yorum yazarken sadece **“Ne”** veya **“Ne yapar”** sorusunun cevabını yazar.
- o Oysa ne sorusunun cevabı çoğu zaman kodun kendisidir.
- o Aşağıdaki kötü örnekte yazılan “ne yapıyor” sorusunun cevabı (eğer işlev adı da zaten uygun verilmişse) okuyucunun çok az işine yarar.

```
// Benzetim yapar c = benzetimYap(int a, int b);
```
- o **O** halde **“Ne”** ve **“Ne yapar”** sorularının cevapları zaten koddan anlaşılıyorsa bunun üstüne bir de yorum yazmak faydasız, boşuna bir tekrar olabilir.
- o **“Ne”** sorusuna cevap veren yorumlar sadece büyük bir kod bloğunun ne yaptığını açıklamak için kullanılmalıdır.

Kaliteli Kodlama

Yorumda Ne Yazılır (devam)

- o **Tecrübeli bir programcı:** Aşağıdaki ve benzeri soruların cevaplarını yorum olarak yazar:
 - Bu kısmın anlamı nedir?
 - Amacı nedir?
 - Ne işe yarar?
 - Neden (bu kod neden var veya neden bu işi yapıyor)?
 - Neyi? Neye? Nereye? Nereden?
 - Hangi?
 - Ne zaman?
 - Neye göre?
 - Nasıl?
- o **“Neden”** sorusuna cevap veren yorumlar en tercih ettiğimiz yorumlardır. Bu tip yorumlar kodun amacının açıklandığı yorumlardır. Bu tip yorumların değişme olasılığı da az olduğu için (çünkü kod değişir ama neden pek değişmez) bu yorumların bakımı da daha az emek ister.
- o İyi bir örnek:

```
// Verilen iki sayı arasında abc algoritmasına göre en yakın
// değeri hesaplar. Bu değer ekranda yer bulma için gereklidir.
c = benzetimYap(int a, int b) ;
```

Kaliteli Kodlama

🌀 Yorumda Ne Yazılır (devam)

- o Yukarıdaki listede en sonda söylediğimiz **“Nasıl?”** sorusunun cevabını yorumlarda yazıp yazmamak biraz tartışma konusudur.
- o **“Nasıl” sorusuna cevap yazmamayı savunan teze** göre kod her zaman değişebilir bu yüzden de nasıl sorusunun cevabı da sürekli değişir, bakımı zordur. Bu tür yorumlarda yorum çürümesi, güncelliğini yitirmesi dediğimiz olaya sık rastlanır. Üstelik anlaşılır ve sade yazılmış bir kod bloğunun zaten ta kendisi **“Nasıl?”** sorusunun bizzat cevabıdır, o halde iyi bir kodda **“Nasıl”** sorusunu cevaplayacak yorumlara da gerek yoktur.
- o **“Nasıl” sorusuna cevap yazmayı savunan tez** ise kodun karmaşıklığında neyin nasıl yapıldığının anlaşılmasının güçlüğünü işaret eder ve nasıl sorusunun cevabının kodun minik bir özeti olarak yorum satırlarında bulunması gerektiğini söyler.
- o **Bu kitabın yazarına göre** kodu anlaşılır yazınız, ancak açıklama gereği duyarsanız **“Nasıl”** sorusunun cevabını da yorum olarak yazabilirsiniz..
- o Tabii ki yorum yazmak iyi bir alışkanlıktır ancak yorum yazmanıza gerek olmayacak kadar açık ve sade kod yazmak enfes bir şeydir.
- o Son olarak yazdığınız yorumların mutlaka kod ile uyumlu, doğru ve güncel olması gerektiğini belirtelim ve bunu anlatan güzel bir söz ile bu konuyu sonlandıralım.
"Yorumlar yalan söyler ama kod yalan söylemez."

Ron Jeffries

Kaliteli Kodlama

🌀 Kodunuzun İçinde Geleceğe Mektuplar Yazın

- Çok değil 6 ay sonra kodunuzu başka birinin yazdığı koddan ayırt edemeyeceksiniz.
- Bu yüzden kodunuzda o an önemli bulduğunuz noktaları not etmekten çekinmeyin. Örneğin yorum satırlarında karışık bir algoritmayı hangi mantıkla gerçekleştirdiğinize dair küçük bir not gelecekte sizi veya bir arkadaşınızı çok rahat ettirecektir.
- Yorumlarla ilgili tavsiyeler:
 - Değişkenler göz önüne alındığında her değişken bildirimi veya tanımı yaptığınızda açıklayıcı bir kaç satır yorum yazınız.
 - Ana kod bloklarının başında açıklayıcı bir yorum yazınız.
 - Ana noktalarda kod akışını tarif eden yorumlar yazınız.
 - Kabullenme veya varsayımlar yaptıysanız o noktada bunu not ediniz. (Zira bu varsayımlar yanlış olabilir veya ileride değişebilir.)
 - Hata çözdüyseniz o noktada bunu not ediniz. (Böylece tekrar aynı hataya düşmezsiniz).
 - Hüner, ustalık, hile veya ince ayar gerektirmiş olan noktalarda bunu not ediniz.

Kaliteli Kodlama

☞ Macar Notasyonu (Hungarian Notation) Hakkında

- o Macar notasyonu düzenli kod yazmayı teşvik ettiği için bir zamanlar çok moda oldu. Macar notasyonunun özü değişken, sabit, işlev vb adlarında özellikle asıl isme ön ekler getirerek tip bilgisi, anlam bilgisi gibi özelliklerin ilgili isme bakar bakmaz anlaşılmasını sağlamaktır.
- o Macar notasyonunun birkaç dezavantajı vardır:
 - Önüne birçok ek harf getirilmiş olan bir kelime okunduğu zaman çok anlamsız olmaktadır.
 - Ayrıca programda bir değişkenin tipi değişirse Macar notasyonu ile yazılmış o değişkeni tüm dosyalarda aratıp yeni ismiyle değiştirmek bayağı zaman alıcı bir iştir.
- o Son yıllarda Macar notasyonunun sağladığı kolaylıklar kalktı ve dezavantajları daha çok göze batar oldu. Şöyle ki birçok modern kod yazma editörü (IDE) zaten imleci ilgili değişken, işlev, vb kelimelerin üzerine getirdiğimizde bize birçok faydalı bilgiyi sağlamaktadır. Bu bilgiyi bir de değişken, vb adlarında tekrar etmenin anlamı yoktur.
- o Bu sebeple Macar notasyonunun katı ve sayıca fazla kuralları yerine biraz daha esnek, kullanıcı dostu, kullanıma ve amaca uygun ve faydalı isimlendirme gelenekleri kullanılmaya başlamıştır.

Kaliteli Kodlama

🌀 Macar Notasyonu (Hungarian Notation) Hakkında (devam)

- o Eğer kullanılmak istenirse Macar notasyonunun sadece Semantik (Anlam katan) ekleri ve değişkenlerin scope'unu (kapsamını) bildiren ekleri kullanılabilir.
- o Tip bilgisi veren eklerinin kullanılmasına artık gerek yoktur.
- o Örneğin bir hafıza alanı büyüklüğü için bc (buffer count) önekini kullanabilirsiniz ama değişkenin karakter bir değer içerdiğini belirtmek için c önekini kullanmaya gerek yoktur.
- o Nesne yönelimli programlamada üye değişkenler için “m” önekinin kullanılması scope (kapsam) belirlemek için iyi bir örnektir.

Kaliteli Kodlama

Macar Notasyonu (Hungarian Notation) Hakkında (devam)

Önek (Prefix)	Veri tipi (Data type)
b	boolean.
f	flag (bayrak).
by	byte veya unsigned char.
c	char.
dw	DWORD, double word veya unsigned long.
fn	fonksiyon, işlev.
h	handle.
i	int (tamsayı).
l	Long.
n	short int.
p	işaretçi.
s	string.
sz	ASCIIZ null sonlanmış karakter dizisi.
w	WORD unsigned int.
x,y	short, iki kooordinat ekseni için kullanılır.

Örnek: Macar notasyonu kullanarak yazılmış birkaç değişken adı.

lpzName //long pointer
//to zero-terminated
//string
lpctr //long pointer to
//constant string
dwRange//Double Word
hWnd //Handle

Macar Notasyonu eklerine örnekler tablosu

Kaliteli Kodlama

🌀 Karmaşıklığı Artırmayın

- o Bir kodu test etmek bazı zamanlarda aynı kodu yazmaktan biraz daha kompleks bir iş olur.
- o Eğer siz kodu yazarken zekânızın tüm kıvılcımlarım, bildiğiniz tüm karmaşık teknikleri kullanırsanız büyük ihtimalle testi yazarken bu zeka pırıltısına veya daha fazlasına ulaşamayacaksınız.
- o Bu sebeple kodunuzu mümkün olduğunca basit ve anlaşılır yazmaya özen gösterin.
- o Büyük ihtimalle yazdığınız karmaşık koda daha sonra bir o kadar daha kod eklenecektir.
- o Bu durumda işin içinden çıkmak gerçekten de zor olacaktır.
- o Bu sebeple mümkün olduğunca anlaşılır, basit, halk tabiri ile üç kağıt ve oyunlar içermeyen sade kodlar yazınız.

Kaliteli Kodlama

☞ Süper Zekice Kod Yazmaya Çalışmayın

- o Genellikle süper zeki insanlar değişik cin teknikler kullanarak çok hızlı çalışan veya işi çok az satırda halleden kodlar yazabilmektedirler.
- o Ancak daha sonra muhtemelen şu iki sorun olacak:
 - Kodu değiştirmek gerektiğinde kod anlaşılmadığı için bir sürü özel durum gerektiren daha uzun parçalar eklenecektir.
 - Kod anlaşılmadığı için kodu test eden testçi (hatta geliştiricinin kendisi) çok daha fazla zaman harcayacaktır.

Hata ayıklamak kod yazmaktan iki kat daha zordur. Bu yüzden, kodu yapabildiğiniz en zekice şekilde yazarsanız, tanım olarak, muhtemelen hatalarını ayıklayabilecek kadar zeki olmayacaksınız.

Brian W. Kernighan

Herhangi bir ahmak bir bilgisayarın anlayabileceği kodu yazabilir, iyi programcılar (ise) insanların anlayabileceği kodlar yazarlar.

Martin Fowler

Anlaşılabilirlik kuralı: Anlaşılabilirlik zekilikten daha önemlidir.

Eric S. Raymond, Unix Felsefesi üzerine

Kaliteli Kodlama

☞ Yazdığınız Kodu Sevin Ama Aşık Olmayın

- o Özeleştiriyi yapma zamanı geldi. Biz yazılımcılar iletişim konusunda süper yetenekli insanlar değiliz. Matematiksel zekâmızın tepede olması duygusal zekâmızın en tepede olmasını gerektirmiyor.
- o Yazılımcıların yaptığı en büyük hatalardan biri de kodlarına duygusal olarak bağlanmak, herhangi bir eleştiri kabul etmemektir.
- o Yazdıkları kodlarda bir yerlerde bir hatanın varlığı söylendiği zaman kendi kişiliklerine karşı bir eleştiri hatta bir saldırı olarak kabul eden, gerçekten iyi yazılımcı ama sosyal alanda çok zayıf bireyler olan birçok programcı var.
- o “İletişim kurmak”, “eleştiriye açık olmak” ve “iyi bir dinleyici olmak” gibi konularda eksiklerimiz olabileceğini (hatta olduğunu) baştan kabul edip bu eksikleri gidermeye çalışmalıyız. Bu çabanızın yararını göreceksiniz.
- o Kodunuz anne-babanız, en sevdiğiniz takım veya sevgiliniz değildir, canlı bile değildir, kodunuz sorunluysa hemen terk edin, değiştirin. Tıpkı arabanız gibi.
- o İyi bir yazılımcı olmak sadece program yazabilmek değildir, birçok özellik daha ister.
- o Aşağıdaki listeye bakın ve siz de kendinizce maddeler ekleyin. Sonra da bu yönlerinizi geliştirmeye çalışın.
- o İyi yazılımcı = İyi programcı+iyi dinleyici+iyi bir dilbilimci+mütevazi+.....

Kaliteli Kodlama

🌀 Kendi Algoritmalarınız Yerine Varsa Evrensel Standart Algoritmalar Kullanın

- o Artık yazılım mühendisliği başlangıç aşamalarını çoktan geçti. Birçok algoritma ve haberleşme protokolü standartlaşmış durumdadır. (Algoritmadan kastımıza örnek verecek olursak sıkıştırma algoritmaları, ses, video işleme algoritmaları, sıralama ve tarama algoritmaları olabilir.)
- o Standart haline gelmiş algoritmalar binlerce hatta milyonlarca kişi ve cihaz tarafından kullanılmış, sağlamlıkları test edilmiş algoritmalar.
- o Literatürü iyi takip etmeli ve çalıştığınız konuda bir algoritmanın geliştirilip geliştirilmediğini öğrenmelisiniz. Eğer ilgilendiğiniz konuda standart bir algoritma var ise mümkünse bunu kullanınız.
- o Müşterinize direneceğiniz birkaç konu var ise bunlardan biri de bu konuda taviz vermemektir. Elbette müşterinin çok özel ve mecburi ihtiyaçları olabilir, ancak standart yöntemlerle bu algoritma ve protokol ihtiyaçlarını halledebiliyorsanız lütfen yapınız.
- o Bir keresinde bir müşteri TCP/IP algoritması için alternatif çözüm istemişti. Böyle garip isteklerden kaçınıp müşteriyi ikna ediniz. Tekerleği yeniden keşfetmek iyi değildir ve sanıldığı kadar kolay değildir.

Kaliteli Kodlama

☞ Ekran Kullanıcı İçin Bir Bulmaca Değildir

- Kullanıcı arabirimi veya ekran düzeni kullanıcının işini kolaylaştırdığı ölçüde başarılıdır.
- Kullanıcı arabirimi hazırlarken başka bölümlerden insanlara denettirin ve yorumlarını alın. Ancak bu yorumları alırken asla karşılık vermeyin, kendinizi savunmaya geçmeyin. Sadece yorumlarını alın ve daha sonra bu yorumlardan faydalanıp tasarımınıza çekidüzen verin.
- Ekran düzeni hem teknik bilgi hem de sanatsal bilgi gerektiren bir iştir. Sanat yönetmeni çalıştırmak çoğu şirket için bir lükstür ancak alternatif çözümler vardır. Örneğin bu işi en iyi yapan sanat yönü gelişmiş ve teknik olarak deneyimli personelinizi bütün GUI'leri denetlemek için görevlendirebilirsiniz. GUI gözden geçirme toplantıları yapabilirsiniz.
- Kullanıcıya geribesleme yapın. Örnek verirsek, bir sistemin kullanıcıyı ilgilendiren bir davranışı sistemin modu değişince etkileniyorsa bu mod değişikliğini kullanıcıya bildirin.
- Bu hem kullanıcı için yararlı hem de test kolaylığı için gereklidir.
- Kullanıcının programı kullanabilmek için aklında tutması gereken işlemleri mümkün olduğunca azaltın.

Kaliteli Kodlama

☞ Ekran Kullanıcı İçin Bir Bulmaca Değildir (devam)

- o Ekran tasarımının 8 altın kuralı (Shneiderman prensipleri)
 - Tutarlılık ve uyum için çabalayın,
 - Usta kullanıcılar için kısa-yollar sağlayın,
 - Geribesleme ile bilgilendirme yapın,
 - (Başlangıç, gelişme ve) kapanışı olan diyaloglar oluşturun,
 - Kullanıcı hatasını önleme ve basitçe hatayı ele alma hizmeti sunun,
 - Kolayca işlemleri geri alma imkânı sunun,
 - Kontrolün kullanıcıda olduğu hissini uyandırın,
 - Kullanıcının kısa-sürelili hafıza yükünü azaltın (belleğinde tutması gerekenleri azaltın)
- o Kullanıcı arayüzü olan programlar için kullanıcının çok az bilgisayar bilgisi olduğunu, herhangi bir kullanım kitapçığını okumayacağını hatta yardım menüsünden faydalanma zahmetini bile göstermeyeceğini kabul etmek iyi bir fikirdir.
- o Şu üç pratik kural faydalı olabilir:
 - Program kullanıcıyı en az şaşırtacak şekilde çalışmalıdır,
 - Kullanıcı için yapılacak bir sonraki adım ve nasıl vazgeçileceği (buradan nasıl çıkılacağı) her zaman açık ve anlaşılır olmalıdır,
 - Program kullanıcıyı uyarmadan kullanıcının ahmakça bir şey yapmasına izin vermemelidir.

Kaliteli Kodlama

🌀 Kodunuz Centilmence ve Nazikçe Çöksün

- Çökmeyen kod diye bir şey yoktur. Uzun uğraşlar sonucunda ve titizce yazılmış tüm çalışan programlar eninde sonunda bir gün oluşan bir sorun yüzünden çökerler.
- Ancak bunu yaparken kaba olmak veya centilmen olmak arasında fark vardır. Bunu sağlamak zor değildir.
- Program çöktüğü zaman log'lar tutmalı, hatanın yeri ve kaynağı hakkında bilgi verici mesajlar göstermelisiniz.
- Çok kez kodlayıcıların bir istisna (exception) oluştuğunda kötü bir şey olduğunu kullanıcıya bildirmek yerine ilgili hata mesajını hasıraltı ettiklerini gözledik.
- Bu şekilde birkaç suistimal edici kullanımdan sonra kodda hata bulmak ve ayıklamak neredeyse imkânsız hale gelir.
- Bu kanser olduğunu kabul etmeyip tedavi olup iyileşebilecekken bile tedavi olmamaya benzer.
- Program çalışırken oluşabilecek tüm hataları gerçekçi bir biçimde kabullenmek ve hatadan minimum zarar ve en az acı ile kurtulacak şekilde hatayı işlemeliyiz.

Kaliteli Kodlama

☞ Kodunuz veya Projeniz Çökecekse Bir An Önce Çöksün

- o Mutlaka kodunuza hata tespit edici rutinler koyun. Kendinizi mutlaka test edin.
- o Kodunuz eğer sorunlu ve çökecekse sizin ellerinizdeyken çöksün.
- o Sahaya gittiğinde çöken bir kod size çok daha pahalıya patlayacaktır.
- o Sahada hata ayıklamak inanılmaz zor ve de ayrıca pahalı bir iştir.
- o Normalde olmaması gereken ama yine de olasılık dahilinde olan sorunların kontrolünü kodunuzda yapın.
- o Bir fonksiyona girdiğinizde parametreler limitler dahilinde mi gelmiş kontrol edin.

Onarım Kuralı: Çökecekseniz (yani kodunuz çökecekse) gürültülü ve mümkün olduğunca erken çökün.

Eric S. Raymond, Unix Felsefesi üzerine

Kaliteli Kodlama

🌀 Kodunuz veya Projeniz Çökecekse Bir An Önce Çöksün (devam)

- o Projelerde de aynı durum vardır.
- o Eğer bir projede bir aksaklık var ise bunu ne kadar erken tespit ederseniz o kadar iyidir.
- o Projeniz iptal edilse bile bu projenizin başlarında olsun. Proje sahaya sürüldükten sonra iptal edilen bir projenin getireceği zararı düşünmek bile istemezsiniz.

“Batacaksan erken bat” felsefesi çok önemlidir. (Kısa iş süreçleriniz varsa problemi kısa zamanda tespit edebilirsiniz.) Projede kısa süreli iterasyonlar ile ilerlerken kazara yarım milyon doları batırmak gerçekten çok ama çok zordur.

Coding Horror internet sitesinden

Kaliteli Kodlama

🌀 Koddaki Bir Sorunun Oluşma İhtimali Varsa O Sorun Mutlaka Oluşur

- o Kod yazarken genellikle hepimiz açık kalan noktalar hakkında iyimser tahminlerde bulunur ve sorunun oluşma ihtimali az deriz.
- o Belki eskiden gerçekten de böyle davranmak doğrudur. Çünkü ihtimaller azdı.
- o Ancak günümüzün hızlı bilgisayarları ve yoğun kullanıcı potansiyeli ile koddaki sorunlar hemen her zaman kısa sürede etkilerini göstermektedir.
- o Eğer bir sorunu düzeltmez ve hasıraltı edersek o sorun mutlaka gelip bizi sonradan vurur.

Kaliteli Kodlama

☞ Bu Tablo, Bu Kod veya Bu Algoritma Artık Değişmeyecek Demeyin Değişir

- o Projede tablolar, kodlar, algoritmaların değişeceği gerçeği aslında bilimsel bir teoriye de dayanmaktadır. Bu teorinin adı “Sıfır-Bir-Sonsuz” teorisidir (İngilizce adıyla “Zero One or Infinity” (ZOI)).
- o Bu teoriye göre bir şeye ya “hiç izin verilmez” ya “sadece bir kez izin verilir” ya da “sonsuz kez izin verilir”.
- o Yani bir şeye iki kez izin verelim diye bir durum yoktur, boşuna limit koymayın nasıl olsa dahası da gelecektir, sonsuz kez izin verin diye düşünülür.

"Bir şey olabiliyorsa o şey olur."

Murphy Kanunu

"Kullanılan bir programda (mutlaka) değişiklikler (modifikasyonlar) yapılacaktır. Bir program değiştirildiğinde (modifiye edildiğinde), eğer birileri aktif olarak bu durumu engellemek için çalışmıyorsa, karmaşıklığı artacaktır."

Software Entropy, Lehman, 1985

Kaliteli Kodlama

Bir Hata Oluştı Hatası

- o Lütfen “Bir hata oluştu” şeklinde hata mesajları göstermeyin.
- o Ayrıca “Hata: 1357911” şeklinde kullanıcının “Bu hata kodu asal sayılar dizisine ne kadar da benziyor” çıkarımından başka bir çıkarım yapamayacağı hata mesajları da göstermeyin.

Kaliteli Kodlama

🌀 Test İçin Tasarım Yapın

- o Tasarım yaparken her an “Ben bunu nasıl test ederim” diye düşünün.
- o Test edemeyeceğiniz bir ürün tasarlamayın.
- o En büyük sorun kodlamak değil yazdığınız kodu doğrulamaktır.

"Şeffaflık Kuralı: Gözden geçirme ve hata ayıklamayı kolaylaştırmak için görünürlük temelli tasarım yapın."

Eric S. Raymond, Unix Felsefesi üzerine

Kaliteli Kodlama

🌀 Tasarım Desenlerini Öğrenin ve Kullanın

- o Genelde tasarım desenleri denilince nesne yönelimli programlama (object oriented programing) akla gelse de nesne yönelimli olmayan programlama çeşitlerinde de birçok tasarım deseni vardır.
- o Tasarım deseninden kasıt önceden defalarca denenmiş ve belli bir sorunun çözümü için uygun görülen yöntemlerden bahsediyoruz.
- o Roma'yı yeniden keşfetmeye gerek yoktur, yazılım mühendisliği artık bir sorunun nasıl daha iyi çözülebileceğine ilişkin yöntemleri oluşturmuş ve oluşturmaktadır. Bu örnekleri öğrenip kullanmanız önerilir.

Kaliteli Kodlama

Kokan Kod (Code Smell)

- o Kod içerisinde bir sorun olabileceğine dair belirtiler bulunan koda kokan kod denir. İngilizce “Code Smell” denilen bu olaya Türkçe’de “Bu kodda bir bit yeniği var” şeklinde bir karşılık uygun düşer.
- o Kokan kod ile hatalı kodu birbirine karıştırmamak lazımdır. Kokan kod bir sorunun kesin olarak varlığını işaret etmez ancak sorun olma ihtimalinin yüksek olduğunu gösterir. Bazen bilinçli olarak yazılmış bir kod da koku yayabilir ama gerçekte hiçbir sorunu yoktur.
- o Kokan kod genellikle incelenip yeniden düzenlenmeye (refactoring) çalışılır. Eğer büyük bir sorun varsa tasarım gözden geçirilir.
- o Bazen performans sorunları veya anlaşılabilirlik gibi sebeplerle kokan koda izin bile verilebilir, ancak yine de kokan kod adından da anlaşılacağı gibi iyi bir incelemeyi hak eden, sorun içermeye ihtimali çok yüksek olan bir kod parçasıdır.
- o Her geçen gün sorunlu olabilecek kodları tahmin etme ipuçları keşfedilmektedir.
- o Zaman zaman bu ipuçları birbirleri ile de çelişebilmektedir ve üstelik yukarıda da belirttiğimiz gibi kodun kesin sorunlu olduklarını da bildirmezler.

Kaliteli Kodlama

🌀 Kokan Kod (Code Smell) (devam)

- Yine de kokan koda bazı örnekler vermek gerekirse:
 - **Aşırı fazla veya hiç konmamış yorumlar:** Derlenebilir kodların yorum satırları içinde kaybolduğu kodlar veya koddan rahatlıkla anlaşılabilen konular için fazla fazla yorum satırları ilgili kodda bir kalite sorunu olabileceğini gösterir.
 - **Tekrar eden kod:** Üç kez veya daha fazla tekrar eden aynı kod alanını bir fonksiyon içinde toplamak gerekir. Bu yapılmamış ise kodda bir kalite sorunu olabilir.
 - **Aşırı benzer işlevler:** Birbirlerine çok benzeyen, hatta aynı işi yapan birden fazla fonksiyonlarınız varsa bu fonksiyonları overload etmeyi denemelisiniz.
 - **Fazla uzun işlev veya sınıf:** Fazla uzun işlev veya sınıflar yazılım kodlanırken üzerinde fazlaca düşünülmendiğini, özenilmediğini, işin parçalara ayrılmadığını akla getirir.
 - **Başka sınıfın işini yapmaya çalışan sınıf:** Eğer yazdığınız bir class, başka bir class'ın metotlarını çok sık kullanmaya başladıysa, belki de iki class'ı birlikte düşünmelisiniz.
 - **Başka sınıfın işine karışan sınıf:** Eğer yazılmış bir class başka bir class'ın işine karışmaya başlamışsa, kendisini ilgilendirmeyen işler yapmaya kalkıyorsa, kodunuz kokuyor dikkat!!!
 - **Tembel Sınıf:** Malumunuz çok az kullanılan sınıf. Az kullanıldığı için ya hiç sorun çıkarmaz, ya da çıkardığında sorunun nereden geldiğinin anlaşılması zor olabilir.

Kaliteli Kodlama

☞ Kokan Kod (Code Smell) (devam)

- **Gereksiz kompleks yapı:** Alt tarafı bir formdaki verileri veritabanına kayıt edecek bir yapı için bin satır kod, onlarca sınıf, vs yazarak pireyi deve yapmışsanız kodunuz kokuyordun
 - **Çok fazla parametre:** Eğer bir işlevin 5-6 parametreden daha fazla parametresi varsa bu işlevin tasarımında bir sorun var demektir. Parametreler birbirleri ile ilişkili ise onları bir yapı veya nesne içinde toplayarak bir parametre olarak geçiriniz ya da tasarımınızı gözden geçiriniz.
 - **Aşırı genelleme:** Tasarım yaparken birçok şeyi düşünmek iyidir ama her şeyi düşünmeye çalışmış bir tasarım başarısızlığa uğrar.
 - **Dev koşul bloğu:** Eğer nesne yönelimli programlama yapıyorsanız ve koşul bloklarınız (if- else veya switch-case gibi bloklar) giderek dev gibi büyüyorsa nesne yönelimli programlamanın tasarım desenlerinden birini kullanmayı deneyebilirsiniz.
- o Yukarıdaki listeye sürekli yeni maddeler eklendiği için ve zamanla ve teknoloji ile bazıları şekil değiştirip bazıları listeden çıktığı için listeyi uzatmakta bir anlam görmüyoruz.
 - o İnternette fazlası ile kokan kod saptama önerileri bulabilirsiniz.